

Statistical Machine Learning: Assignment 3

Pranshu Prakash
Roll Number: 2024426

April 2026

1 Question 1: Classification using Ridge and Lasso Regression

1.1 Methodology and Math

The MNIST dataset (classes 0, 1, 2) was flattened, normalized, and reduced to $p = 10$ dimensions using Principal Component Analysis (PCA). The data was centered, and the covariance matrix S was computed as:

$$S = \frac{X_c^T X_c}{N - 1}$$

Targets were one-hot encoded as +1 for the target class and -1 otherwise.

Ridge Regression (L2 Penalty): Shrinks weights smoothly. It uses a closed-form solution:

$$W = (X^T X + \lambda I)^{-1} X^T Y$$

Lasso Regression (L1 Penalty): Drives unimportant weights to exactly zero, performing feature selection. It optimizes:

$$\min_W \frac{1}{2n} \|XW - Y\|_2^2 + \lambda \|W\|_1$$

1.2 Outputs and Results

- **Data Shape:** Train: (18623, 784), Test: (3147, 784)
- **Best Hyperparameters:** Ridge best $\lambda = 10$, Lasso best $\lambda = 0.0001$
- **Test Accuracy:** Ridge Accuracy = 95.00%, Lasso Accuracy = 95.00%

1.3 Plots and Explanations

Observation: For Ridge, the MSE stays flat because the large training dataset makes the $X^T X$ matrix values dominate the small λI penalty. For Lasso, as λ increases beyond 10^{-2} , the L1 penalty aggressively forces weights to zero, causing the error to spike.

Observation: At small λ , all 10 features are used. As λ increases, the model drops features until all coefficients become zero.

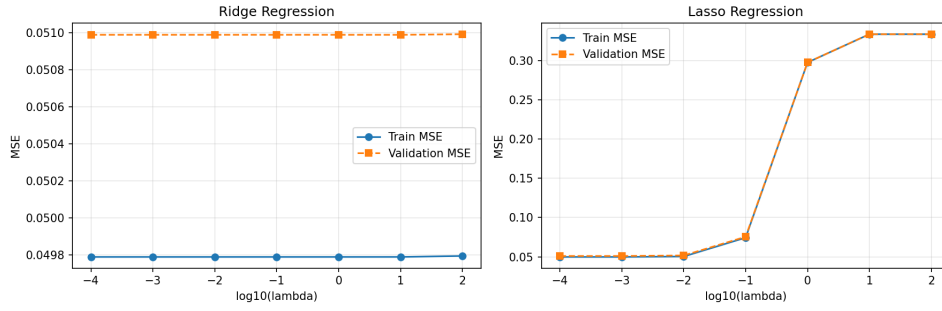


Figure 1: MSE vs Lambda for Ridge and Lasso

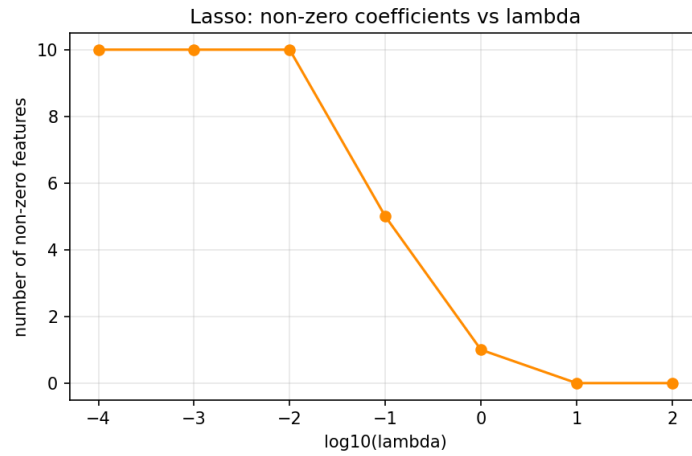


Figure 2: Lasso Non-zero Coefficients vs Lambda

Observation: Ridge coefficients stay relatively flat. Lasso coefficients take sharp paths toward zero, showing clear feature selection.

Observation: Both train and test errors drop as the number of PCA dimensions (p) increases. The sharpest drop is between $p = 2$ and $p = 5$, showing that 2 dimensions are not enough to separate the classes.

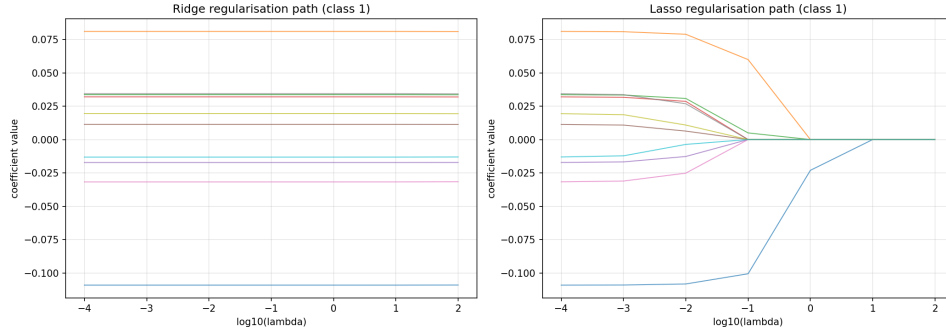


Figure 3: Regularization Paths for Class 1

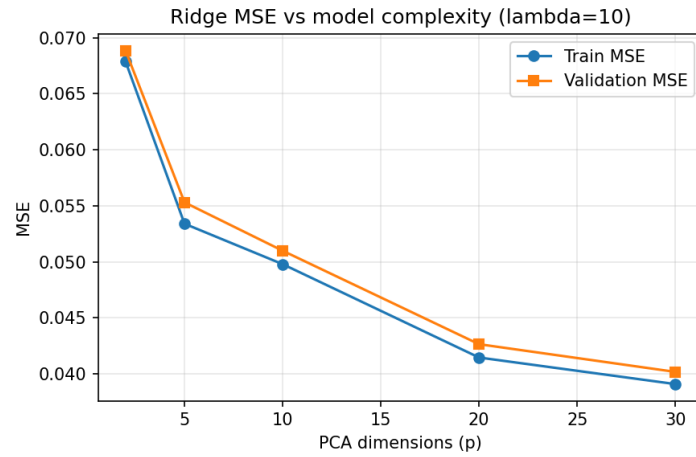


Figure 4: Ridge MSE vs Model Complexity

2 Question 2: Decision Trees and Ensemble Methods

2.1 Methodology and Math

A decision tree was built with a maximum of 3 leaves (2 splits). Splits were chosen to minimize the weighted Gini impurity:

$$Gini(S) = 1 - \sum_{k=1}^K p_k^2$$

$$Gini_{weighted} = \frac{|S_L|}{|S|} Gini(S_L) + \frac{|S_R|}{|S|} Gini(S_R)$$

Threshold Choice: The median (50th percentile) of the feature values was used as the split threshold at each node. The median evenly divides the data points and is robust against extreme outlier values in continuous PCA space, making it computationally efficient and statistically balanced.

2.2 Outputs and Results

1. Single Decision Tree

- **Splits:** Root split on Feature 0 (threshold -0.4768). Second split on Feature 0 (threshold -3.352).
- **Overall Accuracy:** 70.29%
- **Class Accuracies:** Class 0: 99.29%, Class 1: 69.69%, Class 2: 43.41%

Analysis: The single tree struggles heavily with Class 2. Because it only has 3 leaves, the third leaf becomes a catch-all for mixed data, leading to poor accuracy for the remaining class.

2. Bagging (5 Trees)

- **Average OOB Error:** 0.3017
- **Overall Accuracy:** 70.26%
- **Class Accuracies:** Class 0: 99.29%, Class 1: 69.78%, Class 2: 43.22%

Analysis: Bagging barely improves the accuracy. Because all 5 bootstrapped trees use the same dominant PCA features, they pick similar splits (highly correlated trees), so averaging them provides little benefit.

3. Random Forest

- **Best k parameter:** $k = 8$ gave the lowest OOB error (0.3347).
- **Overall Accuracy ($k = 8$):** 70.38%

Analysis: By forcing the splits to choose from a random subset of k features, Random Forest forces the trees to be structurally different (decorrelated). While standard theory suggests $k \approx \sqrt{p} = 3$, a higher $k = 8$ works better here because the tree is very shallow (only 2 splits); a low k risks hiding the most important features entirely.

3 Question 3: Regression using Decision Stumps

3.1 Methodology and Math

We applied a regression decision stump (a tree with exactly 1 split) to the Fashion-MNIST dataset. The stump splits the data by finding the feature and threshold that minimizes the Sum of Squared Residuals (SSR):

$$SSR = \sum_{i \in L} (y_i - \bar{y}_L)^2 + \sum_{i \in R} (y_i - \bar{y}_R)^2$$

Where $\bar{y}_L$ and $\bar{y}_R$ are the mean target values in the left and right regions. For Bagging, 5 stumps were trained on bootstrap datasets, and their predictions were averaged:

$$\hat{y}(x) = \frac{1}{5} \sum_{i=1}^5 h_i(x)$$

3.2 Outputs and Results

- **Data Shape:** Train: (18000, 784), Test: (3000, 784)
- **Single Stump Details:** Split on Feature 2, Threshold 1.4508.
- **Single Stump Test MSE:** 0.3629
- **Bagging Average OOB MSE:** 0.3462
- **Bagging Test MSE:** 0.3594

3.3 Plots and Explanations

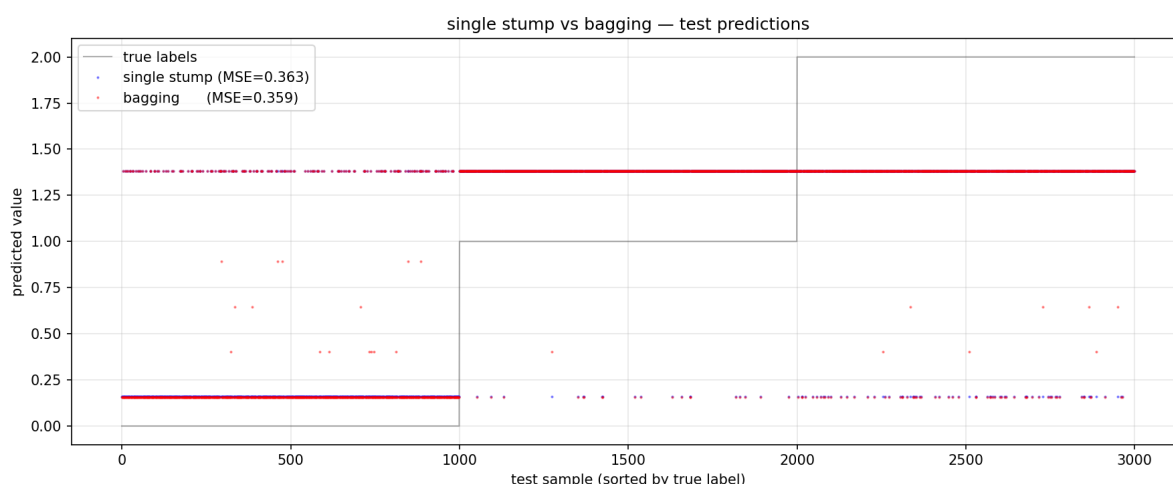


Figure 5: Single Stump vs Bagging Predictions

Explanation of the Difference:

- **Single Stump:** A single decision stump only splits the data once, so it can only predict two possible values (the mean of the left split and the mean of the right split). In the plot, this looks like a rigid horizontal line with only two levels.

- **Bagging:** The bagged model averages 5 different stumps. Because each stump is trained on different bootstrap data, they pick different thresholds. Averaging them creates a smoother prediction with multiple intermediate values, which fits the true data better and slightly lowers the MSE (from 0.3629 to 0.3594).